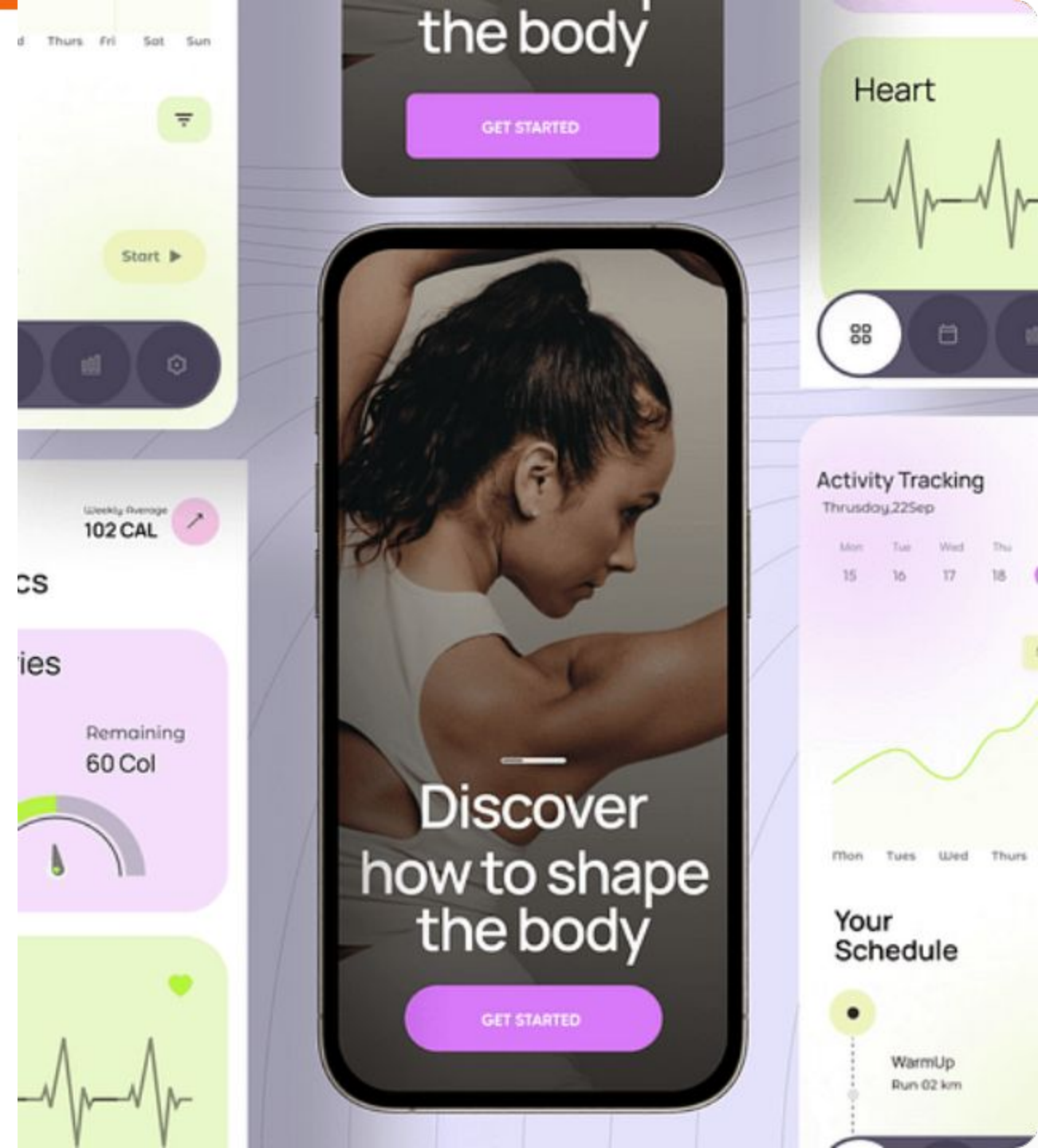


Final Project PASD

FitLogic

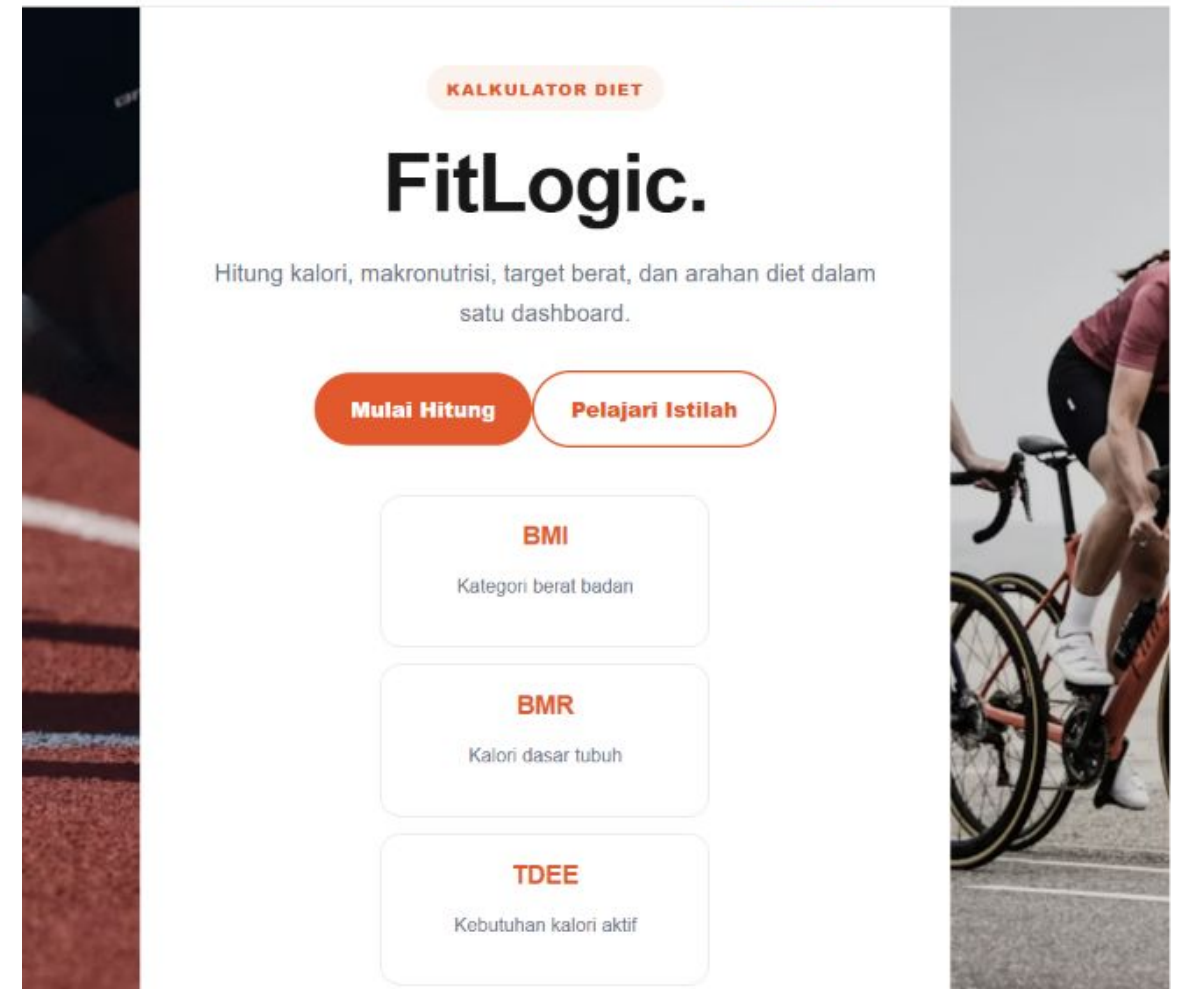
Sistem Rekomendasi Diet Berbasis Data

Aplikasi kalkulator diet interaktif dengan visualisasi dashboard yang mudah dipahami, dibangun dengan pendekatan **Object-Oriented Programming (OOP)** menggunakan HTML, CSS, dan JavaScript.



Ringkasan **Eksekutif**

- ❗ **Masalah Pengguna:** Banyak orang masih bingung cara menghitung kebutuhan kalori harian. Akibatnya, target diet sering dibuat asal-asalan dan kurang sesuai dengan kondisi tubuh.
- ❗ **Kekurangan Alat Biasa:** Kalkulator diet yang umum biasanya hanya menampilkan angka, seperti BMI atau kalori, tanpa penjelasan yang mudah dipahami.
- ✅ **Solusi FitLogic:** FitLogic membantu menghitung BMI, BMR, TDEE, target kalori, dan makronutrisi secara otomatis berdasarkan data pengguna.
- ✅ **Arahan Hasil:** FitLogic juga memberikan Diet Match Score, estimasi progres berat badan, dan dashboard agar pengguna lebih mudah memahami langkah diet yang perlu dilakukan.



Latar Belakang & Tujuan

Latar Belakang

Sekarang banyak orang mulai peduli dengan pola hidup sehat, terutama soal berat badan dan pola makan. Tapi, kebutuhan kalori setiap orang itu berbeda-beda, tergantung umur, jenis kelamin, tinggi badan, berat badan, dan aktivitas harian.

Diperlukan aplikasi yang memudahkan perhitungan kalori dan target diet. FitLogic dibuat agar hasil perhitungan tidak hanya berupa angka, tetapi juga ditampilkan dalam dashboard yang lebih jelas dan mudah dipahami.

Tujuan Aplikasi

- 1. Akurasi Perhitungan:** Membantu pengguna menghitung kebutuhan energi dasar dan kalori harian berdasarkan data pribadi.
- 2. Penentuan Arah:** Membantu pengguna mengetahui apakah targetnya termasuk menurunkan berat badan, menaikkan berat badan, atau mempertahankan berat badan.
- 3. Panduan Visual:** Hasil perhitungan ditampilkan dalam bentuk dashboard, lengkap dengan makronutrisi, Diet Match Score, estimasi progres, dan arahan diet.

Gambaran **Fitur Utama**



Target Personal

Sistem memproses input pengguna seperti data demografis dasar, metrik antropometri (berat & tinggi), dan menetapkan target berat badan spesifik secara interaktif.



Kalkulator Diet

Mesin utama aplikasi yang menjalankan algoritma untuk menghitung Body Mass Index (BMI), Basal Metabolic Rate (BMR), hingga Total Daily Energy Expenditure (TDEE).



Hasil & Skor

Menyajikan visualisasi data medis ke dalam format yang mudah dipahami, memberikan skor kesesuaian diet secara real-time.



Dashboard Aksi

Merangkum hasil akhir ke dalam satu layar pintar. Menampilkan *Diet Match Score*, trend estimasi waktu target, dan rekomendasi nutrisi.

Data Input Pengguna

Aplikasi ini membutuhkan input langsung dari pengguna untuk melakukan personalisasi data diet. Berikut adalah daftar variabel utamanya:

Nama Variabel	Tipe Data	Penggunaan dalam Sistem
Umur	Numerik (Integer)	Variabel vital untuk perhitungan BMR.
Jenis Kelamin	Kategorikal	Menentukan konstanta pada rumus BMR pria/wanita.
Tinggi & Berat Saat Ini	Numerik (Float)	Menjadi basis kalkulasi BMI dan BMR dasar.
Target Berat Badan	Numerik (Float)	Menentukan arah diet (Loss/Gain) dan kalkulasi waktu.
Tingkat Aktivitas	Kategorikal	Faktor pengali (Multiplier) untuk menemukan TDEE.
Intensitas Program	Kategorikal	Menentukan defisit atau surplus kalori spesifik harian.

Rumus Dasar 1: BMI

Body Mass Index (BMI) digunakan untuk mengkategorikan rasio berat badan dan tinggi pengguna sebagai titik awal analisis.

$$\text{BMI} = \frac{\text{Berat (kg)}}{\text{Tinggi (m)}^2}$$

Kategori Skor BMI:

Kurus: Kurang dari 18.5

Normal: 18.5 - 24.9

Gemuk: 25.0 - 29.9

Obesitas: Lebih dari 30



Rumus Dasar 2: **BMR**

Basal Metabolic Rate (BMR) adalah kebutuhan kalori absolut untuk menunjang fungsi dasar tubuh saat istirahat total. Rumus Mifflin-St Jeor digunakan di aplikasi ini.

Laki-laki

$$10W + 6.25H - 5A + 5$$

Variabel yang berlaku:

W = Berat badan dalam satuan kg.

H = Tinggi badan dalam satuan cm.

A = Umur pengguna dalam tahun.

Perempuan

$$10W + 6.25H - 5A - 161$$

Dampak Pemilihan:

Perbedaan gender sangat mempengaruhi laju metabolik dasar. Perubahan input gender akan mengubah angka TDEE secara berantai dalam sistem.

Rumus Dasar 3: **TDEE**

Total Daily Energy Expenditure (TDEE)

Hasil total kalori yang terbakar dalam sehari, memperhitungkan aktivitas fisik pengguna.

$$\text{TDEE} = \text{BMR} \times \text{Faktor Aktivitas}$$

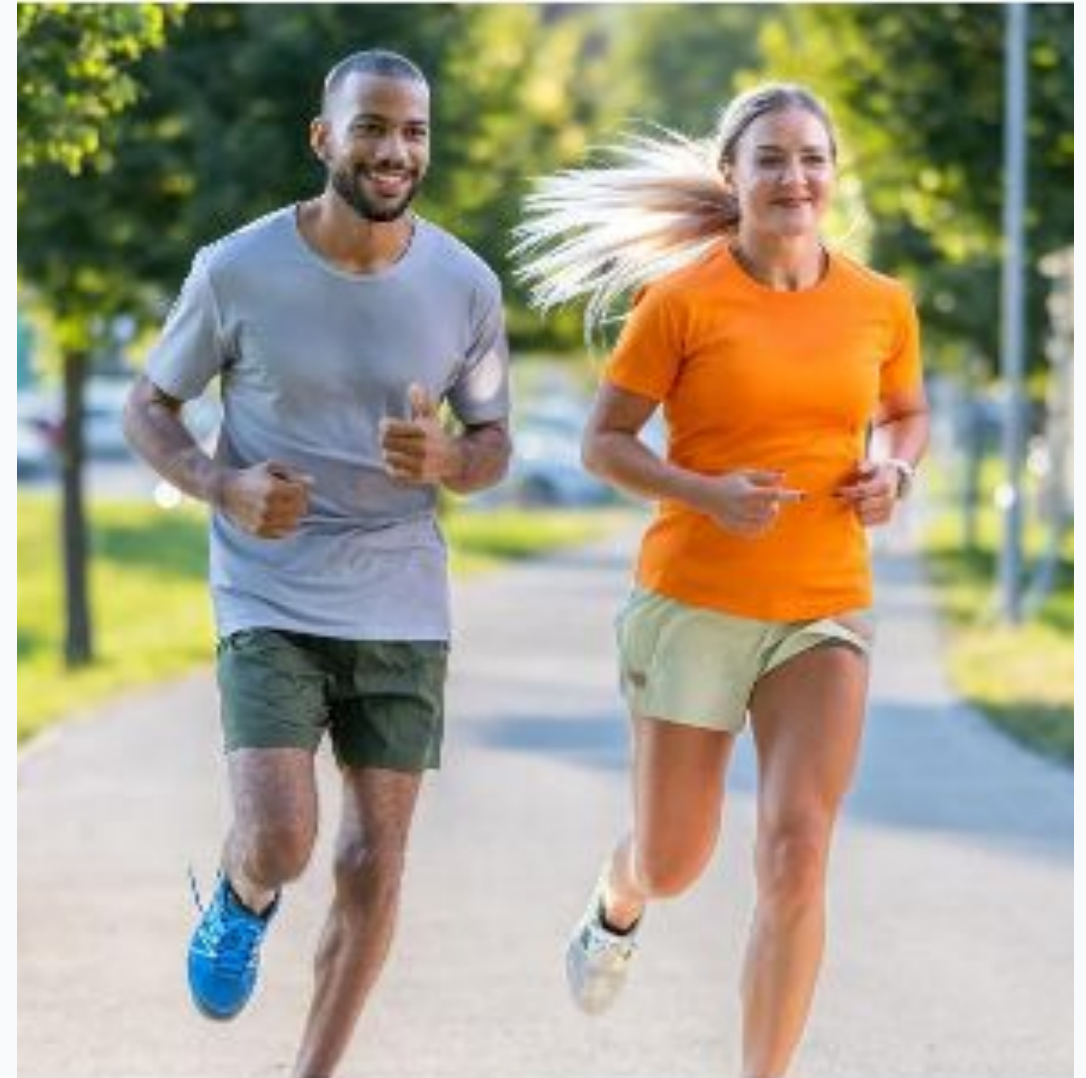
Pengali (Multiplier) Aktivitas Fisik:

Sedentary (Sangat Jarang): 1.20

Ringan (1-3x / minggu): 1.375

Sedang (3-5x / minggu): 1.550

Tinggi (6-7x / minggu): 1.725



Target Kalori & Diet Match Score

1. Algoritma Target Kalori

Sistem memanipulasi TDEE berdasarkan tujuan akhir:

- **Turun Perlahan:** TDEE dikurangi 250 kkal
- **Turun Cepat:** TDEE dikurangi 500 kkal
- **Maintain (Tetap):** Sama dengan TDEE
- **Naik Perlahan:** TDEE ditambah 250 kkal
- **Naik Cepat:** TDEE ditambah 400 kkal

2. Diet Match Score

Sistem cerdas kami tidak hanya memberikan angka, namun melakukan serangkaian validasi otomatis terhadap keamanan diet:

- Mengevaluasi apakah target BMI berada di area sehat.
- Memastikan asupan target tidak berada di bawah BMR.
- Mencegah skenario penyesuaian berat yang ekstrem.

Rumus **Estimasi Waktu Target**

$$\text{Minggu} = \frac{|\text{Berat Sekarang} - \text{Target}|}{\text{Rate Program}}$$

Contoh Sample Ade:

Berat sekarang: 75 kg | **Target:** 69 kg

Selisih berat: 6 kg

Intensitas: Turun perlahan (0.25 kg/mgg)

Estimasi: 6 / 0.25 = **24 minggu**

Intensitas	Rate (kg/minggu)
Turun Perlahan	0.25
Turun Cepat	0.50
Naik Perlahan	0.25
Naik Cepat	0.35
Maintain	0 / stabil

Sample Output: **Skenario Pengguna**

1.811

Target Kalori Harian (kkal)

Diet Match Score: 100%

Profil Input "Ade"

Menggunakan input sebagai berikut: Laki-laki, Umur 20 Tahun, Tinggi 170 cm, Berat 75 kg.

Tujuan: Menurunkan berat ke 69 kg (Turun Perlahan) dengan aktivitas minim (Sedentary).

Hasil Mesin Kalkulasi:

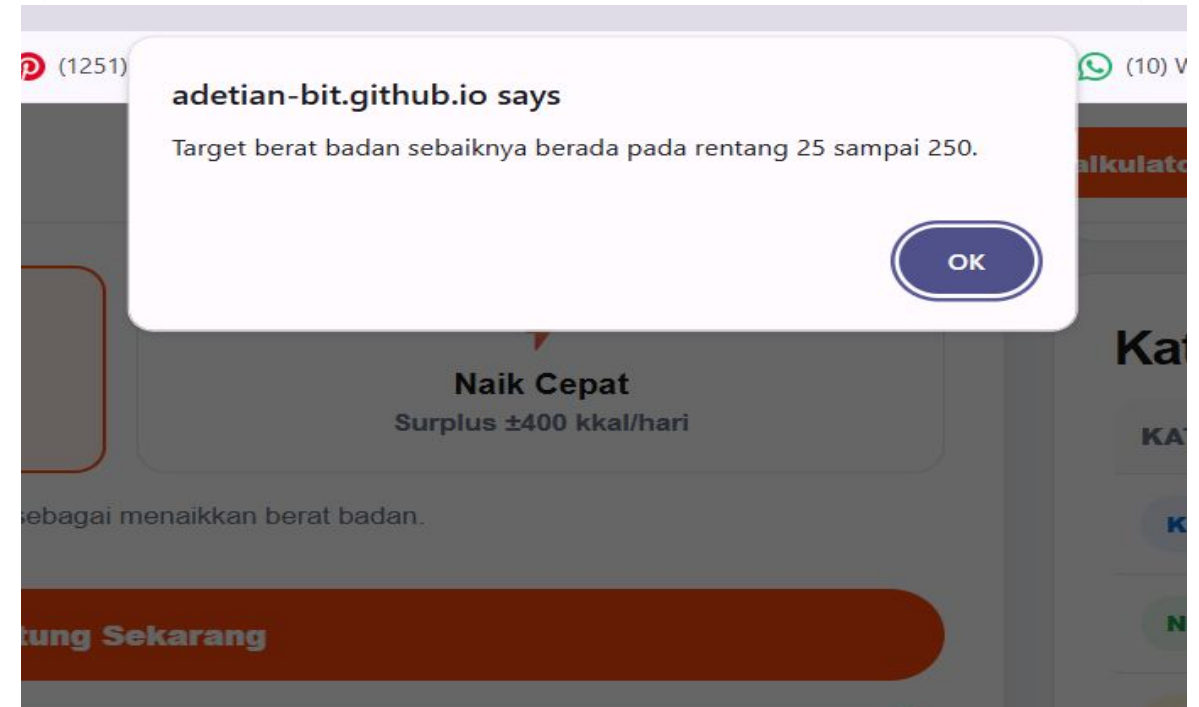
- BMI: **26.0** (Kategori Gemuk)
- BMR: **1.718 kkal**
- TDEE: **2.061 kkal**
- Estimasi waktu tercapai: **~24 minggu**

Validasi Data & Penanganan Error

Untuk menjaga integritas perhitungan dan menghindari crash, aplikasi memvalidasi form sebelum memproses instance OOP.

- ❌ **Aturan Tipe Data:** Semua kolom pengisian numerik tidak dapat menerima karakter alfabet.
- ❌ **Batasan Usia:** Dibatasi keras pada rentang 10 hingga 100 tahun.
- ❌ **Batasan Antropometri:** Tinggi badan (100-230 cm) dan Berat Badan (25-250 kg).

Contoh Peringatan: "Target berat badan sebaiknya berada pada rentang 25 sampai 250 kg."



Alur Pengembangan Teknis

2. Styling CSS

Mendesain antarmuka visual modern, merancang komponen Card interaktif dan tata letak Grid.

4. Export & Deploy

Mengekspor struktur menggunakan skrip Python otomatis dan mendistribusikan ke GitHub Pages.

1. Struktur HTML

Menyusun fondasi kerangka web semantik untuk Kalkulator, Hasil, dan Glosarium.

3. Logika JS OOP

Mengimplementasikan class untuk kalkulasi data independen tanpa bergantung pada server backend.

Pengembangan Aplikasi: Screenshot Kode

SS 1: CSS Dashboard

Bagian desain tampilan website: warna, card, grid, tombol, dashboard.

```
HTML_CSS = r"""
:root{ --primary:#ff4b0b; --primary2:#ff7a1a; --bg:#f4f5f7; --surface:#ffffff; --text:#171717; --muted
}
*{box-sizing:border-box}
html{scroll-behavior:smooth}
body{ margin:0; background:var(--bg); color:var(--text); font-family:Inter,Arial,Helvetica,sans-serif;
}
button,input,select{font-family:inherit}
.topbar{ height:68px; background:#fff; border-bottom:1px solid var(--border); display:flex; align-item
}
.logo{ color:var(--primary); font-size:26px; font-weight:1000; letter-spacing:--5px;
```

SS 2: Body HTML

Struktur halaman: navbar, kalkulator, hasil, dashboard, glosarium.

```
HTML_BODY = r"""
<header class="topbar"><div class="logo">FitLogic</div><div class="nav"><button class="active" data-page="
</header><main id="home" class="page active" style="padding:0;min-height:auto"><div class="hero"><div clas
</main><section id="calc" class="page"><h1 class="center">Kalkulator Diet</h1><p class="sub center">Lengka
</section><section id="result" class="page"><h1 class="center">Hasil Perhitungan</h1><p class="sub center"
</section><section id="dash" class="page"><h1 class="center">Dashboard</h1><p class="sub center">Visualisa
</section><section id="gloss" class="page"><h1 class="center">Glosarium Gizi</h1><p class="sub center">Pen
</section><footer class="footer"><div class="logo">FitLogic</div><p>© 2025 FitLogic. Dibuat untuk edukasi.
</footer>
```

SS 3: JavaScript OOP

Contoh class OOP: User / CalorieCalculator / Dashboard.

```
JS_USER = r"""
class User {
  constructor({name, age, gender, height, weight, targetWeight, activity}) {
    this.name = name || "Pengguna";
    this.age = age;
    this.gender = gender;
    this.height = height;
    this.weight = weight;
    this.targetWeight = targetWeight;
    this.activity = activity;
```

SS 4: Python Export HTML

Cell Python yang menulis HTML_TEMPLATE menjadi file .html.

```
HTML_TEMPLATE = f"""<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>FitLogic Pro V2 - Dashboard Edukatif</title>
<style>
{HTML_CSS}
</style>
```


Implementasi OOP JavaScript (1/3)

Class User

Bertanggung jawab menampung seluruh state profil dan metode dasar untuk entitas individu.

```
JS_USER = `
class User {
  constructor({name, age, gender, height, weight, targetWeight, activity}) {
    this.name = name || "Pengguna";
    this.age = age;
    this.gender = gender;
    this.height = height;
    this.weight = weight;
    this.targetWeight = targetWeight;
    this.activity = activity;
  }
  get heightM() {
    return this.height / 100;
  }
  get bmi() {
    return this.weight / (this.heightM * this.heightM);
  }
  get targetBMI() {
    return this.targetWeight / (this.heightM * this.heightM);
  }
}
```

Class CalorieCalculator

Fokus khusus dalam komputasi fisika dasar tubuh (Metabolik dan aktivitas harian).

```
JS_CALORIE_CALCULATOR = `
class CalorieCalculator {
  constructor(user, intensity) {
    this.user = user;
    this.intensity = intensity;
  }
  calculateBMR() {
    const u = this.user;
    if (u.gender === "male") {
      return (10 * u.weight) + (6.25 * u.height) - (5 * u.age) + 5;
    }
    return (10 * u.weight) + (6.25 * u.height) - (5 * u.age) - 161;
  }
  calculateTDEE(bmr) {
    return bmr * this.user.activity;
  }
  getAdjustment() {
    if (this.user.direction === "loss") return this.intensity === "fast" ? -500 : -250;
    if (this.user.direction === "gain") return this.intensity === "gainFast" ? 400 : 250;
    return 0;
  }
  getProgramLabel() {

```

Implementasi OOP JavaScript (2/3)

Class MacroRecommendation

Memecah target kalori harian menjadi distribusi gizi spesifik berbasis penelitian standar.

```
JS_MACRO_RECOMMENDATION = `
class MacroRecommendation {
  constructor(targetCalories, weight, direction) {
    this.targetCalories = targetCalories;
    this.weight = weight;
    this.direction = direction;
  }
  calculate() {
    let proteinFactor = this.direction === "loss" ? 1.8 : this.direction === "gain" ? 1.7 : 1.6;
    let protein = this.weight * proteinFactor;
    protein = Math.min(protein, this.targetCalories * 0.35 / 4);

    let fat = Math.max(this.weight * 0.6, this.targetCalories * 0.23 / 9);
    fat = Math.min(fat, this.targetCalories * 0.35 / 9);

    let carbCalories = this.targetCalories - (protein * 4) - (fat * 9);

    if (carbCalories < this.targetCalories * 0.18) {
      carbCalories = this.targetCalories * 0.22;
      const remaining = this.targetCalories - carbCalories;
      protein = Math.min(protein, remaining * 0.58 / 4);
      fat = Math.max(0, (remaining - protein * 4) / 9);
    }
  }
}
```

Class DietProgress

Mensimulasikan garis waktu pencapaian dari titik berat saat ini menuju target akhir.

```
JS_DIET_PROGRESS = `
class DietProgress {
  constructor(user, intensity) {
    this.user = user;
    this.intensity = intensity;
  }
  getRate() {
    if (this.user.direction === "loss") return this.intensity === "fast" ? 0.5 : 0.25;
    if (this.user.direction === "gain") return this.intensity === "gainFast" ? 0.35 : 0.25;
    return 0;
  }
  calculate() {
    const absDiff = Math.abs(this.user.targetWeight - this.user.weight);
    const rate = this.getRate();
    return {
      absDiff: absDiff,
      rate: rate,
      estimateWeeks: rate === 0 ? 0 : Math.ceil(absDiff / rate)
    };
  }
}
```

Implementasi OOP JavaScript (3/3)

Class DietMatchScore

Penjaga keamanan parameter. Memberikan "skor sehat" terhadap kombinasi angka yang dibentuk mesin kalkulator.

```
JS_DIET_MATCH_SCORE = `
class DietMatchScore {
  constructor(result) {
    this.result = result;
    this.score = 100;
    this.good = [];
    this.bad = [];
    this.warnings = [];
    this.healthyMinWeight = 18.5 * Math.pow(result.heightM, 2);
    this.healthyMaxWeight = 24.9 * Math.pow(result.heightM, 2);
  }
  evaluateTargetBMI() {
    if (this.result.targetBMI < 18.5) {
      this.score -= this.result.targetBMI < 17 ? 65 : 45;
      this.bad.push("Target berat membuat BMI target masuk kategori kurus. Untuk tinggi ${this.result.height} cm, target se");
      this.warnings.push("Target berat terlalu rendah. Pertimbangkan target minimal sekitar ${Utils.one(this.healthyMinWeig");
    } else if (this.result.targetBMI >= 30) {
      this.score -= 28;
      this.bad.push("Target berat masih membuat BMI target masuk kategori obesitas. Target bisa dibuat bertahap menuju rent");
    } else {
      this.good.push("BMI target masih berada pada rentang yang lebih aman untuk dijadikan tujuan awal.");
    }
  }
}
```

Class FitLogicApp

Orkestrator sistem (Controller). Merangkai alur inisiasi objek, perhitungan berantai, dan interaksi DOM antarmuka.

```
JS_FITLOGIC_APP = `
class FitLogicApp {
  constructor() {
    this.gender = "male";
    this.intensity = "slow";
    this.storage = new StorageManager();
    this.dashboard = new Dashboard(this);
  }
  init() {
    document.querySelectorAll(".nav button").forEach(button => {
      button.addEventListener("click", () => this.openPage(button.dataset.page));
    });

    this.refreshIntensity();

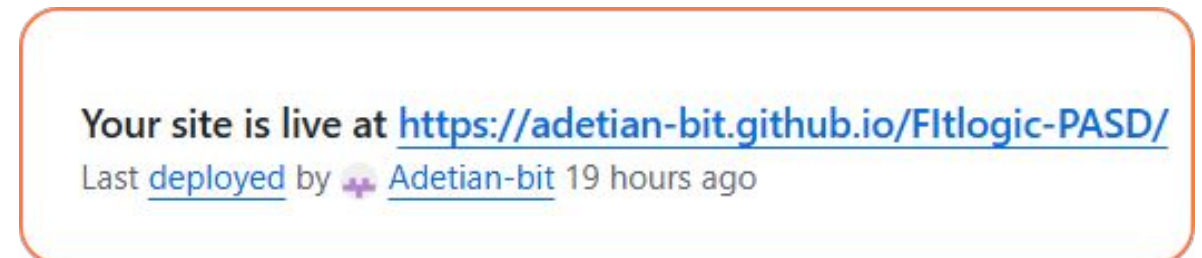
    const saved = this.storage.loadResult();
    if (saved) {
      this.dashboard.renderResult(saved);
      this.dashboard.renderDashboard(saved);
    }
  }
}
```

Deployment via **GitHub Pages**

Sistem ini dirancang murni di sisi *Client-Side*, sehingga tidak memerlukan infrastruktur server ekstensif.

Deployment dilakukan dengan cepat menggunakan platform statis.

-  **Langkah 1:** Mempersiapkan repositori dan mengunggah kode sumber lengkap (HTML, CSS, JS).
-  **Langkah 2:** Melakukan aktivasi fitur "GitHub Pages" melalui opsi *Settings* pada repositori.
-  **Langkah 3:** Menentukan *branch* target (misal: main) sebagai sumber perenderan.
-  **Langkah 4:** Menyimpan perubahan, dan aplikasi web live secara instan dengan URL terenkripsi HTTPS.



Kesimpulan Akhir

FitLogic berhasil mengonstruksi paradigma baru dalam edukasi kesehatan diet digital.

Aplikasi ini merevolusi perhitungan matematis klinis (BMI, BMR, Kalori, Makronutrisi) menjadi informasi praktis yang tertata manis dalam sebuah dashboard personal.

Dengan pemanfaatan murni arsitektur **JavaScript OOP** yang berjalan utuh di browser, FitLogic menawarkan performa kalkulasi tanpa jeda, keamanan terjamin, serta platform yang kokoh dan mudah diskalakan untuk fitur analitik lanjutan di masa depan.

Link Website: <https://adetan-bit.github.io/Fltlogic-PASD/>

